

Yomimasu — MVP Brief for Full-Stack Developer

1. Project overview

Project name: Yomimasu

Domain: yomimasu.com

Product: Japanese graded reader web app

Target users: English-speaking Japanese learners, mainly JLPT N5–N3

Core promise: Master Japanese reading at your level.

Yomimasu helps Japanese learners read short level-based texts. Users can click words or phrases, see instant readings/meanings/grammar help, save vocabulary, track known/new words, and ask AI to explain sentences in context.

The main product is the **reader experience**.

2. MVP goal

Polished but simple validation MVP. This experience:

Open Japanese text

→ read with optional furigana

→ click any word or phrase

→ instantly see reading, meaning, base form, grammar/form, kanji breakdown

→ save or mark word as known

→ track progress

→ ask AI for sentence explanation if needed

3. Visual style

The landing page and app should follow the provided mockup direction:

warm paper-white / white-rosa colors

Japanese-inspired visual style

sakura decorations

clean, calm, premium, modern

not anime

not childish

not generic AI SaaS

The UI should feel polished, but not overcomplicated.

“Master Japanese reading at your level. Reading is one of the most powerful tools for vocabulary, grammar, and comprehension — especially in Japanese.”

Master Japanese reading at your level.

Reading is one of the most powerful tools for vocabulary, grammar, and comprehension—especially in Japanese. Yomimasu makes it easy with instant support every step of the way.

Start Reading →

Go to Dashboard

Browse Library

Try Demo →



Loved by learners worldwide

春のピクニック

日曜日、私は友だちと公園へ行きました。
天気よかったので、みんなで弁当を持て行きました。木の下にシートを敷いて、いろいろな話をしながら、おいしい昼ごはんを食べました。
食べたあと、近くの川を散歩しました。
とても楽しい一日でした。

1 / 6

Aa

食べました

たべました

Reading tabemashita

Meaning ate

Base form 食べる

Save word

Explain

Known words

423

New words

18

Reading streak

8 days

Everything you need to turn reading into real Japanese progress.

Clickable words

Click any word for meanings and forms.

Furigana & readings

See readings above kanji automatically.

Grammar in context

Understand grammar through real examples.

Saved vocabulary

Save words and review them anytime.

Progress tracking

Track your reading and vocabulary growth.

AI sentence help

Get clear explanations for any sentence.

Start reading now with 2 free texts.

Easy, engaging stories. Click to start reading instantly.

Free text

N5 Morning Routine

朝のじゅんぴ

A simple story about a busy morning. Perfect for beginners.

Read now →

Free text

N4 Spring Picnic

春のピクニック

Enjoy a day in the park with friends in this short story.

Read now →

Keep growing with texts at every level.

Discover stories that match your level, interests, and goals.

Browse all →

N5 Daily Life

24 texts

N4 Stories

28 texts

N3 News

20 texts

Travel

18 texts

Work & School

20 texts

See your Japanese level grow.

Stay motivated with clear insights.

Go to Dashboard →

Known words

423

New words

18

Reading time

5h 24m

Reading streak

8 days

Level progress

N5 → N4 → N3

You're here

Next goal

76%

Reading progress

100%

50%

0%

Mon

Tue

Wed

Thu

Fri

Sat

Sun

Vocabulary status

Known

423 (76%)

New

103 (17%)

Review

32 (7%)

Start building real Japanese today.

Jump in, explore stories, and enjoy the journey.

Start Reading →

Go to Dashboard

Browse Library

5. Reduced page structure

To keep the MVP lean, use as few main pages as possible.

Required routes/pages:

/	Landing page
/read/[slug]	Reader page
/app	User dashboard with tabs
/admin	Admin panel with tabs

Avoid unnecessary separate pages.

Maybe pricing page.

maybe library page.

Use tabs/sections instead of many pages where possible.

6. Public landing page

The landing page should be interactive from the start and invite users to start reading immediately.

Required landing page sections:

Hero section

2 free texts ready to click

Reader preview

Feature cards

Explore library preview

Dashboard/progress preview

Final CTA

Important:

No pricing section on the landing page. Pricing only when using premium features (premium texts, more AI explanations, etc)

Hero CTAs:

Start Reading

Go to Dashboard

Browse Library

Try Demo

The landing page should include **2 free texts** that users can click immediately.

Example:

Free Text 1: N5 Morning Routine

Free Text 2: N4 Spring Picnic

All other text categories should be shown under **Explore the Library**, not as immediately free texts.

Dashboard and performance tracking is also free for all users, but naturally limited due to the lack of texts.

7. User access model

For MVP:

Anonymous users can open the 2 free texts.

If they want to save words, track progress, or use dashboard, they need to create an account.

free user limit:

2 free texts

2 AI sentence explanations

Payments/Stripe in the MVP can be discussed

8. Authentication

All users should be able to:

sign up

log in

log out

save vocabulary

track their own reading progress

Email/password or magic link is enough.

Social login is optional.

9. Reader page

The reader page is the most important page.

It should display:

Japanese text

optional furigana toggle ON/OFF

clickable words/tokens

clickable particles

clickable phrases if admin has defined them

sentence selection or sentence highlighting

text metadata: level, word count, known words, new words, estimated reading time

When the user clicks a word/token, a popup should open instantly.

Example popup:

食べました

たべました

Base: 食べる

Meaning: ate

Form: polite past

[Save word] [Mark as known] [Explain sentence]

The popup should support:

reading

base form

main meaning

alternative meanings

part of speech

grammar/form

kanji breakdown

example sentence if available

word/kanji level if available

Example for 友達:

友達

ともだち

Meaning: friend / friends

Part of speech: noun

Kanji:

友 = friend

達 = plural suffix / reach / accomplished

Important: the app should treat 友達 as one vocabulary word in the text, while also showing the individual kanji breakdown. The main saved item should be the full vocabulary word, not the individual kanji.

10. Paragraph/read tracking

When a user marks a paragraph as read, the app should increment the “seen/read count” for words in that paragraph.

Important:

Seeing a word should not automatically make it “known”.

Basic word statuses:

unseen

read/seen

saved

known

11. Japanese tokenization and dictionary engine

This is a core requirement.

The app must not rely only on manually written vocabulary lists.

When an admin adds a Japanese text, the system should process it automatically:

Japanese text

- split into sentences
- tokenize into words/morphemes
- detect surface form
- detect base form / lemma
- detect reading
- detect part of speech
- match to dictionary entry
- generate furigana
- store token data

Example input:

私は友達と公園へ行きました。

Expected tokenization:

私 / は / 友達 / と / 公園 / へ / 行きました / 。

Expected data:

Surface	Base Reading	Type	Meaning
私	私 わたし	pronoun/noun I / me	
は	は は	particle	topic marker
友達	友達 ともだち	noun	friend
と	と と	particle	with / and
公園	公園 こうえん	noun	park
へ	へ へ	particle	direction marker
行きました 行く	いきました	verb	went

Developer should propose the best tokenizer/morphological analyzer, for example:

Sudachi / SudachiPy

Kuromoji

MeCab

other suitable option

Please explain the tradeoffs.

12. Dictionary integration

The app needs a Japanese-English dictionary layer.

Expected sources can include:

JMdict for vocabulary

KANJIDIC2 for kanji

frequency/JLPT datasets if legally usable

The system should support:

word/expression

reading

base form

part of speech

English meanings/glosses

kanji meanings/readings

frequency if available
estimated JLPT level if available
admin override for level/meaning

Important:

Word clicks should not call AI by default.

Word popups should use stored dictionary/token data.

The popup must be instant.

13. Word and kanji levels

If possible, show approximate word/kanji level.

Example:

友 = N4

達 = N3

But this does not need to be perfect in MVP.

If reliable JLPT data is not available, store:

`estimated_level`

and allow admin override.

14. Context-specific meanings

Many Japanese words have multiple meanings.

The app should show the most relevant meaning first, and optionally show alternative meanings.

Example:

行きました

Base: 行く

Primary meaning in this sentence: went

Other meanings: go, proceed, be done, pass

There must be an admin override system for token meanings.

15. Phrase support

Some Japanese expressions should be treated as phrases.

Example:

持って行きました

The tokenizer may split it as:

持って / 行きました

But for learners, it may be better to show:

持って行く = to bring / take along

The admin panel should allow grouping tokens into a phrase and adding phrase-level meaning/explanation.

MVP phrase support can be simple, but the data model should allow this.

16. Particles

Particles should also be clickable.

Examples:

は

を

へ

で

と

が

に

Particle popups should be grammar-style, not normal dictionary-style.

Example:

は

Particle

Topic marker.

In this sentence, it marks 私 as the topic.

17. Furigana

The reader must support furigana above kanji.

Furigana should be optional and toggleable (ON/OFF)

18. AI integration

AI is limited to contextual learning help.

AI is **not** the main dictionary.

Required AI feature:

Explain sentence

When the user select and clicks “Explain sentence,” send to the backend:

selected sentence

paragraph/context if useful

selected word/token if applicable

user level: N5/N4/N3

base language: English

AI should return structured data:

sentence translation

grammar breakdown

selected word explanation if applicable

simple explanation for learner level

1–2 extra examples if useful

Example:

Sentence: 私は友達と公園へ行きました。

Selected word: 行きました

User level: N5

Expected explanation:

は is a particle that marks 私 (me) as the topic.

と is a particle that means “with” and 友達 means friend/s, so 友達と means “with my friend/s.”

へ is a particle that marks direction, 公園 means “park”, so 公園へ means “to the park.”

行きました is the polite past form of 行く (to go), meaning “went.”

So the full meaning is: “I went to the park with my friend/s.”

Technical requirements:

OpenAI API integration

API key stored server-side only

AI explanations cached/saved

reuse existing explanation if already generated

basic rate limiting

structured JSON output preferred

Do not call AI every time for the same sentence explanation if it is already cached.

19. User vocabulary and progress

Users should be able to save words and track progress.

Saved word data should include:

user_id

dictionary_word_id

surface form first seen

base form

reading

primary meaning

source text

sentence where first seen

status: read / saved / known

times read

times clicked

created_at

last_seen_at

Basic dashboard should show:

texts read

words saved

known words

new words

reading streak

reading time

recent texts

recent saved words

level progress: N5 → N4 → N3

The dashboard should include a compact level progress view.

Example:

Level progress: N5 → N4 → N3

Current: N5

Next goal: N4

20. Known/new word tracking per text

For every text, the app should calculate for the logged-in user:

total tokens

unique vocabulary words

read/seen words

new words

saved words in this text

known word percentage

estimated reading time

Example display:

Known words: 82%

New words: 18

Level: N4

Reading time: 5 min

This is an important feature.

21. User dashboard /app

The /app page should contain tabs or sections:

Library

Vocabulary

Progress

The dashboard should show:

available texts

saved vocabulary

known/new/read word stats

level progress

reading streak

recent activity

This avoids building separate pages for library, vocabulary, and dashboard.

22. Admin panel /admin

Admin panel can be simple. It does not need to be beautiful.

Admin should be able to:

- create/edit/delete texts
- assign level: N5/N4/N3
- assign topic/category
- paste Japanese text
- run tokenization/dictionary processing
- review generated tokens
- edit token meaning
- edit reading
- edit base form
- edit part of speech
- group tokens into phrases
- add phrase explanation
- add/edit English translation
- publish/unpublish text

Admin correction is essential because automatic Japanese tokenization/dictionary matching will not always be perfect.

23. Content

For MVP:

- 5 sample texts loaded
- 2 public free texts
- 3 additional texts visible in library/app

Texts will be provided by me.

Later, the app should support many more texts, but the MVP does not need all of them loaded at launch.

24. Suggested tech stack

Preferred stack:

- Next.js
- PostgreSQL / Supabase
- Supabase Auth or similar
- Vercel

OpenAI API

Tailwind CSS or similar

Developer can propose alternatives, but the stack should stay **simple, maintainable**, and easy to hand over.

Important ownership requirement:

Code must be in my GitHub account or transferred to me.

Domain, hosting, database, and API accounts must be owned by me.

Developer gets collaborator access only.

25. MVP exclusions

Not required in this first version:

native mobile app

audio/TTS

pricing page

PDF/EPUB import

all languages

multiple base languages

teacher marketplace

full SRS flashcard system

community features

browser extension

The MVP should be structured so **these can be added later**, but they should not be built now.

26. Deliverables

Developer should deliver:

deployed web app on yomimasu.com or staging subdomain

source code in GitHub

database schema/migrations

basic documentation

admin login

5 sample texts loaded

OpenAI sentence explanation working

Japanese tokenization/dictionary lookup working

reader popup working

saved vocabulary/progress working

handover call/walkthrough

Basic documentation should explain:

how to run locally

how to deploy

environment variables

how to add a text

how tokenization/dictionary processing works

how to correct tokens in admin